



DICT 312 – Application Development

Week 2 Tutorial and Lab Assignment

Due Date for the Submission of the Lab Assignment: 1st

August 2024

Time: 23H00

Lecturer: Dr Olalekan Samuel Ogunleye

**Please take note of the Submission Time. This will not be
extended.**

Before you complete this Tutorial, you must perform the following in addition to what was taught in the class

Go to the following [link](#): and Download Maven 3.9.

Follow the instructions in the [link](#) below to configure your Maven on your computer. This is very important and if you do not follow the instructions, you will not be able to perform the lab successfully and you will be getting error in your implementation of both the tutorial and Lab.

Jakarta Servlet Tutorial: Task Management Application

Overview

In this tutorial, we will create a task management application using Jakarta Servlet, HTML5, and Bootstrap. We will use NetBeans IDE and GlassFish server for our development environment. The application will allow users to create, view, update, and delete tasks.

Prerequisites

- Install NetBeans IDE (version 12.0 or later)
- Install GlassFish server (version 5.0 or later)
- Basic knowledge of Java, HTML, and Bootstrap

Step 1: Setting Up the Development Environment

1. **Download and Install NetBeans IDE:**
 - Visit the [NetBeans download page](#) and download the appropriate version for your operating system.
 - Follow the installation instructions.
2. **Download and Install GlassFish Server:**
 - Visit the GlassFish download page and download the server.
 - Follow the installation instructions.
3. **Create a New Project in NetBeans:**
 - Open NetBeans and select `File > New Project`.
 - Choose `Java with Maven > Web Application` and click `Next`.
 - Name your project (e.g., `TaskManagementApp`) and set the project location.
 - Choose `GlassFish Server` as the server and click `Finish`.

Step 2: Configuring the Project

1. **Add Dependencies:**
 - Open `pom.xml` and add the following dependencies for Jakarta Servlet and JSON processing

```

<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-api</artifactId>
    <version>9.1.0</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.glassfish</groupId>
    <artifactId>jakarta.json</artifactId>
    <version>2.0.1</version>
  </dependency>
</dependencies>

```

1. Create the Servlet

- Right-click on the `src/main/java` folder, select `New > Servlet`.
- Name your servlet `TaskServlet` and set the package to `com.example.taskmanagement`.
- Click `Next` and ensure the URL pattern is set to `/tasks`.
- Click `Finish`.

Step 3: Implementing the TaskServlet

1. Define the Task Model

- Create a new Java class `Task` in the `com.example.taskmanagement` package.

```

package com.example.taskmanagement;

public class Task {
    private int id;
    private String title;
    private String description;
    private boolean completed;

    // Constructors, getters, and setters
    public Task(int id, String title, String description, boolean
completed) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.completed = completed;
    }

    public int getId() {

```

```

        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public boolean isCompleted() {
        return completed;
    }

    public void setCompleted(boolean completed) {
        this.completed = completed;
    }
}

```

Implement CRUD Operations

Update the `TaskServlet` to handle CRUD operations using annotations.

```

package com.example.taskmanagement;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.json.Json;
import jakarta.json.JsonArrayBuilder;
import jakarta.json.JsonObject;
import jakarta.json.JsonObjectBuilder;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

@WebServlet("/tasks")
public class TaskServlet extends HttpServlet {
    private List<Task> tasks = new ArrayList<>();
    private int taskIdCounter = 1;
    @Override

```

```

        protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
            response.setContentType("application/json");
            PrintWriter out = response.getWriter();
            JsonArrayBuilder jsonArrayBuilder = Json.createArrayBuilder();
            for (Task task : tasks) {
                JsonObjectBuilder jsonObjectBuilder =
Json.createObjectBuilder()
                    .add("id", task.getId())
                    .add("title", task.getTitle())
                    .add("description", task.getDescription())
                    .add("completed", task.isCompleted());
                jsonArrayBuilder.add(jsonObjectBuilder);
            }
            out.print(jsonArrayBuilder.build().toString());
            out.flush();
        }
        @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
            String title = request.getParameter("title");
            String description = request.getParameter("description");
            Task newTask = new Task(taskIdCounter++, title, description,
false);
            tasks.add(newTask);
            response.setContentType("application/json");
            PrintWriter out = response.getWriter();
            JsonObjectBuilder jsonObjectBuilder = Json.createObjectBuilder()
                .add("id", newTask.getId())
                .add("title", newTask.getTitle())
                .add("description", newTask.getDescription())
                .add("completed", newTask.isCompleted());
            System.out.print(jsonObjectBuilder.build().toString());
            System.out.flush();
        }
        @Override
        protected void doPut(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
            int id = Integer.parseInt(request.getParameter("id"));
            String title = request.getParameter("title");
            String description = request.getParameter("description");
            boolean completed =
Boolean.parseBoolean(request.getParameter("completed"));
            Task taskToUpdate = null;
            for (Task task : tasks) {
                if (task.getId() == id) {
                    taskToUpdate = task;
                    break;
                }
            }
            if (taskToUpdate != null) {
                taskToUpdate.setTitle(title);
                taskToUpdate.setDescription(description);
                taskToUpdate.setCompleted(completed);
                response.setContentType("application/json");
                PrintWriter out = response.getWriter();
                JsonObjectBuilder jsonObjectBuilder =
Json.createObjectBuilder()
                    .add("id", taskToUpdate.getId())
                    .add("title", taskToUpdate.getTitle())
                    .add("description", taskToUpdate.getDescription())

```

```

        .add("completed", taskToUpdate.isCompleted());
        System.out.print(jsonObjectBuilder.build().toString());
        System.out.flush();
    } else {
        response.sendError(HttpServletResponse.SC_NOT_FOUND, "Task not
found");
    }
}
@Override
protected void doDelete(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    int id = Integer.parseInt(request.getParameter("id"));
    Task taskToDelete = null;
    for (Task task : tasks) {
        if (task.getId() == id) {
            taskToDelete = task;
            break;
        }
    }
    if (taskToDelete != null) {
        tasks.remove(taskToDelete);
        response.setContentType("application/json");
        PrintWriter out = response.getWriter();
        JsonObjectBuilder jsonObjectBuilder =
Json.createObjectBuilder()
            .add("message", "Task deleted successfully");

        System.out.print(jsonObjectBuilder.build().toString());
        System.out.flush();
    } else {
        response.sendError(HttpServletResponse.SC_NOT_FOUND, "Task not
found");
    }
}
}
}

```

Detailed Explanation

In this step, we will implement the CRUD (Create, Read, Update, Delete) operations in our `TaskServlet` class. Here's a detailed explanation of the code:

Task Model

First, we define the `Task` model:

TaskServlet Implementation

Next, we implement the `TaskServlet` class to handle HTTP requests:

Below is what we did in the `TaskServlet` class

1. Class-level Annotations:

- `@WebServlet("/tasks")`: This annotation maps the servlet to the `/tasks` URL pattern. It tells the servlet container to forward requests sent to `/tasks` to this servlet.

2. Servlet Fields:

- `private List<Task> tasks = new ArrayList<>();`: This list stores the tasks.
- `private int taskIdCounter = 1;`: This counter generates unique IDs for new tasks.

3. doGet Method:

- This method handles GET requests. It retrieves the list of tasks and returns them in JSON format.
- `response.setContentType("application/json");`: Sets the response content type to JSON.
- `JsonArrayBuilder jsonArrayBuilder = Json.createArrayBuilder();`: Creates a JSON array builder.
- For each task, a JSON object is created and added to the JSON array.
- `out.print(jsonArrayBuilder.build().toString());`: Sends the JSON array as the response.

4. doPost Method:

- This method handles POST requests. It creates a new task and adds it to the task list.
- `String title = request.getParameter("title");`: Retrieves the task title from the request.
- `Task newTask = new Task(taskIdCounter++, title, description, false);`: Creates a new task with a unique ID.
- The new task is added to the list, and a JSON representation of the task is returned in the response.

5. doPut Method:

- This method handles PUT requests. It updates an existing task.
- `int id = Integer.parseInt(request.getParameter("id"));`: Retrieves the task ID from the request.
- The task is searched in the list using the ID.

- If found, the task's fields are updated, and a JSON representation of the updated task is returned.
- If not found, a 404 error is sent.

6. doDelete Method:

- This method handles DELETE requests. It deletes an existing task.
- `int id = Integer.parseInt(request.getParameter("id"));` Retrieves the task ID from the request.
- The task is searched in the list using the ID.
- If found, the task is removed from the list, and a success message is returned.
- If not found, a 404 error is sent.

It is your responsibility to properly follow this task and by following this detailed explanation, you should clearly understand how each part of the `TaskServlet` class works to handle CRUD operations for the task management application. This will allow you to implement the lab given to you.

Step 4: Creating the Frontend (You Should naturally have the understanding of HTML5 as you have done Web Application Development. Therefore, there is no need to explain this step)

1. Create HTML and Bootstrap Files:

- Create an `index.html` file in the `webapp` directory.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task Management Application</title>
  <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.
css">
</head>
<body>
  <div class="container">
    <h1 class="mt-5">Task Management Application</h1>
    <div id="taskForm">
      <form id="addTaskForm" class="mt-4">
        <div class="form-group">
          <label for="title">Title</label>
          <input type="text" class="form-control" id="title"
placeholder="Enter task title" required>
        </div>
        <div class="form-group">
          <label for="description">Description</label>
          <input type="text" class="form-control"
id="description" placeholder="Enter task description" required>
        </div>
      </form>
    </div>
  </div>
```

```

        <button type="submit" class="btn btn-primary">Add
Task</button>
    </form>
</div>
<div id="taskList" class="mt-5">
    <h2>Task List</h2>
    <ul class="list-group" id="tasks">
    </ul>
</div>
</div>
<script src="https://code.jquery.com/jquery-
3.3.1.slim.min.js"></script>
<script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js
"></script>
<script>
    document.addEventListener('DOMContentLoaded', function () {
        const taskForm = document.getElementById('addTaskForm');
        const taskList = document.getElementById('tasks');
        taskForm.addEventListener('submit', function (e) {
            e.preventDefault();
            const title = document.getElementById('title').value;
            const description =
document.getElementById('description').value;
            fetch('/tasks', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/x-www-form-urlencoded'
                },
                body: `title=${title}&description=${description}`
            })
                .then(response => response.json())
                .then(data => {
                    const taskItem = document.createElement('li');
                    taskItem.className = 'list-group-item';
                    taskItem.textContent = `${data.title} -
${data.description}`;
                    taskList.appendChild(taskItem);
                })
                .catch(error => console.error('Error:', error));
        });
        function loadTasks() {
            fetch('/tasks')
                .then(response => response.json())
                .then(data => {
                    data.forEach(task => {
                        const taskItem = document.createElement('li');
                        taskItem.className = 'list-group-item';
                        taskItem.textContent = `${task.title} -
${task.description}`;
                        taskList.appendChild(taskItem);
                    });
                })
                .catch(error => console.error('Error:', error));
        }
        loadTasks();
    });
</script>
</body>
</html>

```

Step 5: Deploy and Run the Application

1. Deploy to GlassFish Server:

- Right-click on the project in NetBeans and select `Run`.
- NetBeans will deploy the application to the GlassFish server and open the default browser to `http://localhost:8080/TaskManagementApp/`.

2. Testing the Application:

- Use the form to add new tasks.
- Tasks will be displayed in the task list.
- Use browser developer tools to test the PUT and DELETE requests.

Summary Explanation of the Implementation of the Task Management Tutorial

1. Servlet Configuration:

- We use the `@WebServlet` annotation to configure the servlet and map it to the `/tasks` URL pattern.
- The `doGet`, `doPost`, `doPut`, and `doDelete` methods handle GET, POST, PUT, and DELETE requests, respectively.

2. Task Model:

- The `Task` class defines the structure of a task with fields for `id`, `title`, `description`, and `completed`.

3. Handling CRUD Operations:

- `doGet`: Returns a list of all tasks in JSON format.
- `doPost`: Creates a new task and returns the created task in JSON format.
- `doPut`: Updates an existing task and returns the updated task in JSON format.
- `doDelete`: Deletes a task and returns a success message.

4. Frontend Implementation:

- HTML form for adding tasks and a list to display tasks.
- Bootstrap for styling.
- JavaScript for handling form submissions and fetching tasks.

This comprehensive tutorial should help you create a Jakarta Servlet-based task management application. If you have any questions or need further assistance, feel free to ask!

Lab Activity

Due Date: August 1st 2024

Submission Time: 23H00.

Please note that if your network will not allow you to submit, you are required to submit this online on Campus where there is good network connectivity. It is your responsibility to submit on/before the due date. No Excuse will be entertained as to why the submission is not made on time.

Lab Activity Tasks

Develop a book management application using Jakarta Servlet, HTML5, and Bootstrap. This application will allow users to create, view, update, and delete books by following the exact step that was followed in the Tutorial Above.

Book Management Application Overview and Requirements Specification

Overview

The Book Management Application is a simple web-based system designed to manage a collection of books. Users can perform basic CRUD (Create, Read, Update, Delete) operations to manage the book records. The application will be built using Jakarta Servlet, HTML5, and Bootstrap and deployed on the GlassFish server using NetBeans IDE.

Functional Requirements

1. Add New Book:

- Users should be able to add new books to the collection.
- Required fields for a book: title, author, genre, availability status (default to available).

2. View Books:

- Users should be able to view a list of all books.
- The list should display the book title, author, genre, and availability status.

3. Update Book Details:

- Users should be able to update the details of an existing book.
- Users should be able to change the title, author, genre, and availability status of a book.

4. Delete Book:

- Users should be able to delete a book from the collection.

Non-Functional Requirements

1. Usability:

- The application should have a user-friendly interface.
- Bootstrap should be used to ensure the application is responsive and visually appealing.

2. Performance:

- The application should handle CRUD operations efficiently.
- The response time for user interactions should be minimal.

3. Reliability:

- The application should handle errors gracefully and provide meaningful error messages.
- Data should be stored and retrieved accurately.
- 4. **Maintainability:**
 - The code should be well-documented to facilitate future maintenance.
 - The application should follow best practices in coding and design.
- 5. **Compatibility:**
 - The application should be compatible with major web browsers (Chrome, Firefox, Safari, Edge).
 - The application should run on the GlassFish server.

Technical Requirements

1. **Platform:**
 - NetBeans IDE for development.
 - GlassFish server for deployment.
2. **Technologies:**
 - Jakarta Servlet for server-side processing.
 - HTML5 for the structure of web pages.
 - Bootstrap for styling and responsiveness.
 - JavaScript for client-side interactions and AJAX requests.
3. **Dependencies:**
 - Jakarta EE API for Servlet functionality.
 - JSON processing library for handling JSON data.
4. **Data Storage:**
 - In-memory storage for simplicity. In a real-world application, this would typically be a database.

System Architecture

1. **Client-Side:**
 - HTML5 and Bootstrap for the user interface.
 - JavaScript for handling form submissions and AJAX requests to the server.
2. **Server-Side:**
 - Jakarta Servlet for handling HTTP requests and responses.

- In-memory data structures to store book records.
- 3. **Communication:**
 - AJAX for asynchronous communication between the client and server.
 - JSON for data interchange between client and server.

User Interface Design

1. **Home Page:**
 - Display a form to add a new book.
 - Display a list of existing books with options to update or delete each.
2. **Add Book Form:**
 - Fields: Title, Author, Genre.
 - Submit button to add the book to the collection.
3. **Book List:**
 - List each book with details (Title, Author, Genre, Availability).
 - Buttons/Links to update or delete each book.

The Book Management Application will provide a functional and user-friendly system for managing book records by meeting these requirements.

All the best